

NYC Yellow Taxi Trip Explorer

Architecture Design Document

Overview

The system is a four-layer pipeline that turns a raw 7.7M-row TLC parquet file into an interactive urban-mobility dashboard. A one-time **ETL** cleans and enriches trips and loads them into a **PostgreSQL** star schema; a **Flask** service exposes aggregate and detail endpoints (two of which rank results with hand-written algorithms); and a static **HTML/CSS/JS** dashboard renders charts and a zone choropleth. The same Flask process serves both the API and the dashboard, so the whole product runs from a single command.

Layer	Technology	Responsibility
Presentation	HTML/CSS/JS, Chart.js, Leaflet	Filters, charts, map
Application	Flask, custom algorithms	REST API, ranking
Data	PostgreSQL, static GeoJSON	Storage, indexing
ETL	pandas, pyarrow, COPY	Clean, enrich, load

1. Layered Component View

1 · Layered component architecture

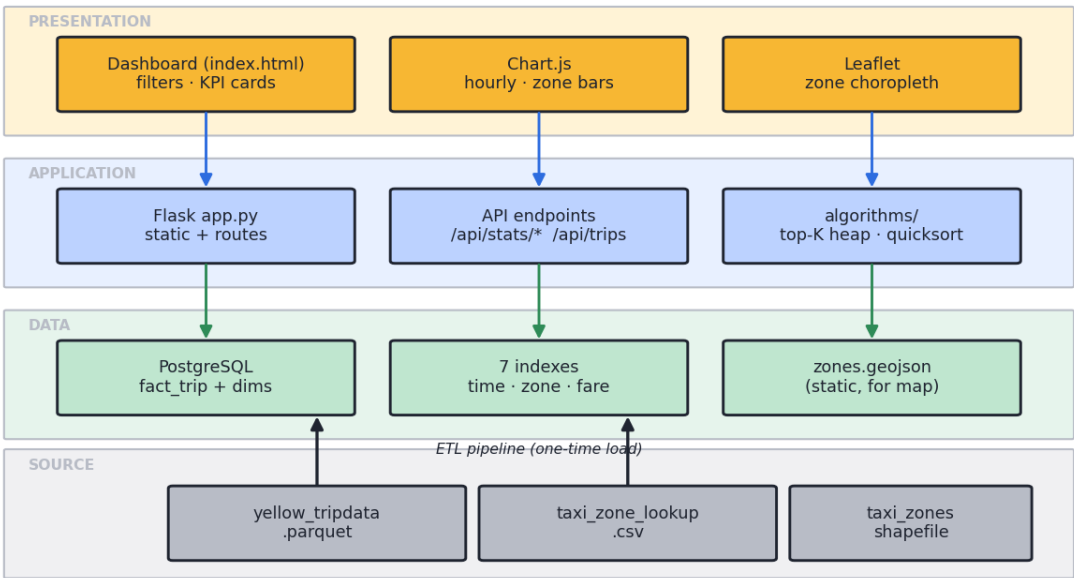


Fig. 1 - Four layers; arrows show runtime calls (top three layers) and the one-time ETL load (bottom).

Presentation is vanilla JS: *app.js* orchestrates filter state, *api.js* calls the backend, *charts.js* drives Chart.js, and *map.js* drives Leaflet. **Application** is Flask: *app.py* serves static files and JSON routes, delegating ranking to *algorithms/*. **Data** is PostgreSQL plus a static *zones.geojson* the map loads directly. **Source** is the three raw TLC files, consumed only by the ETL.

2. ETL Data-Flow Pipeline

2 · ETL data-flow pipeline

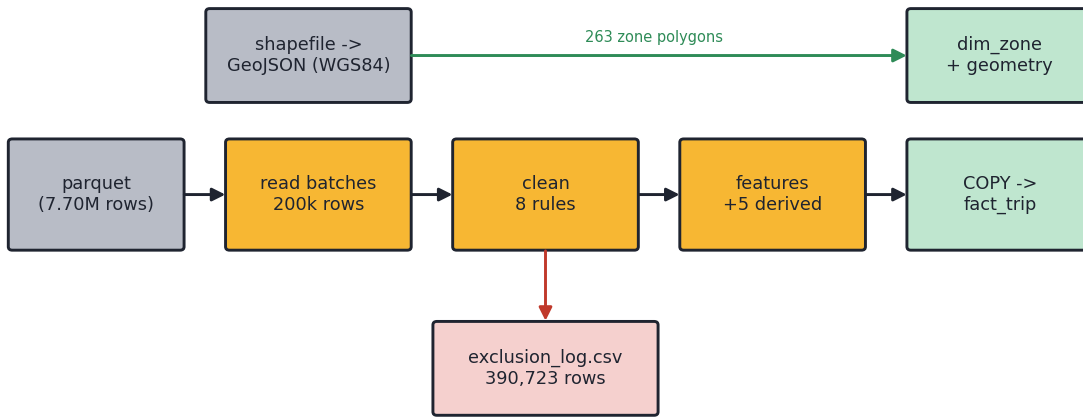
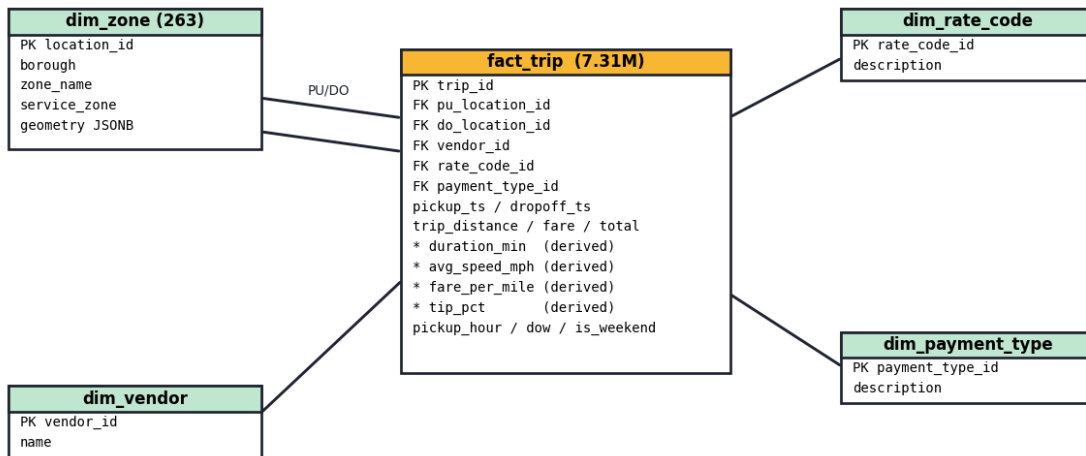


Fig. 2 - Streaming load with an exclusion-log branch and a one-pass zone-geometry branch.

The trip parquet is streamed in 200k-row batches so memory stays bounded. Each batch passes through eight integrity rules (duplicates, missing fields, and physical/logical outliers), then five engineered features are added (duration, average speed, fare-per-mile, tip percent, and temporal keys), and the batch is bulk-loaded with PostgreSQL **COPY**. Rejected rows (**390,723**, 5.1%) are sampled to *exclusion_log.csv* and counted in *etl_exclusion_summary*. In parallel, the zone shapefile is reprojected from EPSG:2263 to WGS84 once and written both to *dim_zone.geometry* and to the static GeoJSON.

3. Database Design (Star Schema)

3 · Entity-relationship (star schema)



* = engineered feature | PK primary key | FK foreign key

Fig. 3 - One fact table referencing four dimensions; engineered features marked with an asterisk.

fact_trip holds one row per trip with foreign keys to four dimensions. *dim_zone* is referenced twice - pickup and dropoff. Engineered features and the temporal keys (hour, day-of-week, weekend) are denormalized onto the fact table so the dashboard's frequent group-bys become single-column scans. Seven B-tree indexes cover the filter columns: pickup timestamp, pickup zone, dropoff zone, hour, fare, distance, and payment type. Foreign keys are nullable so out-of-spec categorical codes can be set NULL without dropping the trip.

4. Runtime Request Flow

4 · Runtime request flow (top-zones endpoint)

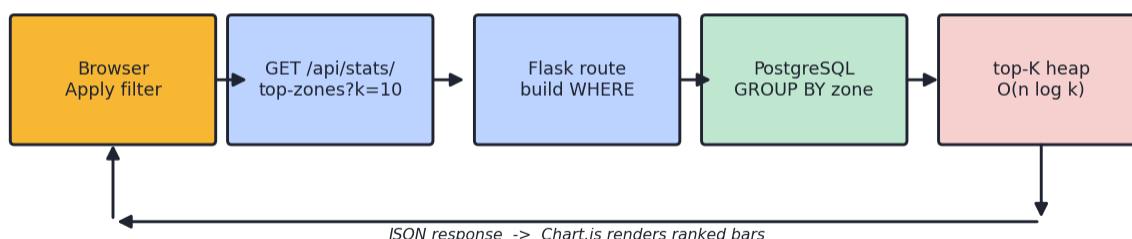


Fig. 4 - A filtered top-zones request, end to end.

When the user applies a filter, the dashboard issues a single GET with the filter as query parameters. The Flask route validates inputs against a whitelist (sort columns and metrics are never interpolated raw) and builds a parameterised WHERE clause. PostgreSQL returns per-zone aggregates over the indexed columns; the route then ranks them with the custom **top-K min-heap** ($O(n \log k)$) rather than SQL ORDER BY/LIMIT, demonstrating the hand-written structure on live data. The JSON response is rendered by Chart.js. The sibling *zone-ranking* endpoint sorts all 263 zones with the custom quicksort to colour the choropleth.

Cross-Cutting Concerns

Security: all SQL is parameterised and sortable columns/metrics are whitelisted, preventing injection.

Performance: chunked COPY load, covering indexes, and denormalized temporal keys keep both load and query fast. **Reproducibility:** the ETL is deterministic and writes a summary (*etl_summary.json*) plus the exclusion log for audit. **Portability:** the API port is configurable (defaults to 5001 to avoid macOS AirPlay on 5000), and the frontend uses relative paths so it works behind any host.